

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules 115

 Bug 15

 Code Smell 100

Tags

▼

Impact

▼

Clean code attribute

▼

Search by name...

🔍

Class names should comply with a naming convention

 Code Smell

Track uses of "TODO" tags

 Code Smell

Deprecated code should be removed

 Code Smell

Cyclomatic Complexity of functions should not be too high

 Code Smell

Constant names should comply with a naming convention

 Code Smell

Unnecessary nullable in final declaration should be removed

 Code Smell

Dependencies should be sorted

 Code Smell

Unused assignments should be removed

 Code Smell

Standard outputs should not be used directly to log anything

 Code Smell

Variables should not be initialized with "null"

 Code Smell

Files should end with a newline

 Code Smell

Standard outputs should not be used directly to log anything

Analyze your code

Adaptability - Modular

Maintainability ⚠️

 Code Smell

 Major ?

 bad-practice

Why is this an issue?

More Info

In software development, logs serve as a record of events within an application, providing crucial insights for debugging. When logging, it is essential to ensure that the logs are:

- easily accessible
- uniformly formatted for readability
- properly recorded
- securely logged when dealing with sensitive data

Those requirements are not met if a program directly writes to the standard outputs (e.g., print). That is why defining and using a dedicated logger is highly recommended.

If you are using Flutter, you can use `debugPrint` or surround `print` calls with a check for `kDebugMode`.

Code examples

Noncompliant code example

```
void doSomething(int x) {  
  // ...  
  print('debug: $x');  
  // ...  
}
```

Compliant solution

```
void doSomething(int x) {  
  // ...  
  debugPrint('debug: $x');  
  // ...  
}
```

or

```
void doSomething(int x) {  
  // ...  
  if (kDebugMode) {  
    print('debug: $x');  
  }  
  // ...  
}
```

or

```
void doSomething(int x) {  
  // ...  
  log('log: $x');  
  // ...  
}
```

Available In:

sonarcloud 

